Pointers, Virtual Functions and Polymorphism

Prepared By Deepali Sonawane, MIT-WPU, Pune

Pointers to Objects

- A pointer can point to an object created by a class.
- Example: employee e;

  employee *ptr = &e;

The pointer ptr is initialized with the address of e.

We can refer to the member functions of an employee in two ways:

1. By using the dot operator (.) and the object. Eg. e.accpet();
2. By using the arrow operator(->) and object pointer. Eg. ptr->accept();

“this” pointer

- When a **member function** is called, it is automatically passed an implicit argument that is a pointer to the invoking object i.e. the object on which the function is called. This pointer is called **this.**
- CPP keeps only **one copy** of each member function and data members are allocated memory for all of their instances. These kind of various instances of data are maintained using **this** pointer.
- **this** pointer is initialized to point to the object for which the member function is invoked.
- **this** pointer is most useful when working with pointers and especially with linked list when you need to reference a pointer.

Example for “this” pointer

- Function definition in class:

```
Number operator>(Number num)// Member function
{
    if(n > num.n)//100 > 50
        return *this;//n1
    else
        return num;//n2
}
```

- Function call in main():

```
Number n1(10), n2(30);
n3 = n1>n2 ; //operator function call
```

Virtual Functions

- Virtual functions are functions that ensure that the correct function is called for an object, regardless of the expression used to make the function call.
- When a function is made virtual, CPP determines which function to use at run time based on the type of object pointed to by the base pointer, rather than the type of the pointer.
- To make a member function virtual, the keyword **virtual** is used in the methods while it is declared in the class definition but not in the member function definition.
- The keyword virtual should be preceded by a return type of the function name.
- The compiler gets information from the keyword virtual that it is a virtual function and not a conventional function declaration.

Pure virtual functions

- The functions which are **only declared** but not defined in the base class are called **pure virtual functions**.
- A function is made pure virtual by preceding its declaration with keyword **virtual** and by post fixing it with **=0**.
- **Syntax: virtual return_type function_name([argument list])=0;**
- A class containing one or more pure virtual functions cannot be used to define an object. The class is therefore only useful as a base class to be inherited into a useable derived class. This class is called as **abstract class**.

Thank you